FIRST TO 4 (CONNECT FOUR)

Name : Adeet Singh

Reg. No. : 25BAI11451

Course : BTECH CSE (AI/ML)

1. Executive Summary

The "First to 4" project is a digital recreation of the classic vertical checkers game. The primary goal was to move beyond a simple two-player local game and create a sophisticated "Man vs. Machine" experience. The final product features a terminal-based interface with a heavy emphasis on Artificial Intelligence (AI) logic, ranging from heuristic-based decision-making to deep-tree searching.

2. Problem Analysis

Connect Four is a "zero-sum" game of perfect information. Unlike games involving dice or cards, there is no luck involved. The challenge in digitizing this was twofold:

1. **Physics Emulation:** Managing the "Gravity" mechanic where pieces must stack on top of each other.
2. **Pattern Recognition:** Detecting 4-in-a-row across four different axes ($0^\circ, 90^\circ, 45^\circ, 135^\circ$) efficiently.

3. Technical Approach

3.1 System Architecture

The application follows a modular procedural structure:

- **Data Representation:** A 6x7 matrix initialized with null values.
- **The Game Loop:** A recursive structure that handles turn-taking, input validation, and board refreshing.
- **Visual Engine:** Uses ANSI escape sequences to render a color-coded board, improving the UX without requiring a heavy Graphical User Interface (GUI) library.

3.2 Artificial Intelligence (The Three Tiers)

- **Level 1 (Stochastic):** Utilizes a random number generator to pick valid columns.
- **Level 2 (Heuristic):** Employs "One-Move Lookahead." It checks if a win is possible for itself or the opponent in the immediate next turn.
- **Level 3 (Minimax):** Implements a recursive search tree. It assigns scores to board states based on "Window Evaluation"—rewarding the AI for creating sequences of 2 or 3 and penalizing it for allowing the player to do the same.

4. Key Implementation Decisions

- **Alpha-Beta Pruning:** To keep the game responsive, I implemented Alpha-Beta Pruning. This optimization ignores branches of the game tree that cannot possibly influence the final decision, reducing the computational load by over 60%.
- **Deepcopy for Simulation:** Because Python passes lists by reference, I used copy.deepcopy to ensure the AI's "imagined" future moves did not alter the actual live game board.

5. Challenges & Solutions

- **The "Out of Bounds" Bug:** Early versions crashed when checking for wins near the edges of the board. I solved this by implementing "boundary-aware" loops that stop checking once the index reaches the board limits.

.

- **Terminal Flicker:** Constant printing caused the screen to flicker. I implemented a "Clear-and-Redraw" strategy using os.system('cls') to provide a smoother visual experience.

6. Conclusion & Future Scope

This project successfully demonstrates the application of classic AI algorithms in a constrained environment. It highlights the power of Python for rapid prototyping. Future iterations could include a **Web API** version using Flask or a **Reinforcement Learning** agent that learns to play better by playing against itself.